



Introduzione a Git – Più di un semplice Backup

Cos'è Git?

Un sistema di controllo di versione distribuito (DVCS).

Perché per la Security?

Garantisce l'integrità del codice (tramite hashing SHA-1), permette l'audit delle modifiche e la gestione sicura delle collaborazioni.

Differenza tra Git e GitHub

Git è lo strumento locale;
GitHub/GitLab sono le piattaforme cloud che ospitano i repository.

L'Architettura dei Tre Stati

Git non salva i file "al volo", ma gestisce il flusso attraverso tre aree principali:



Working Directory

Dove modifichi i file fisicamente.



Staging Area (Index)

Il "limbo" dove prepari i file per il prossimo commit.



Local Repo (.git)

Dove le modifiche vengono salvate permanentemente nella cronologia.

Il concetto di "Snapshot", non differenze

Fotografia dell'intero progetto

A differenza dei vecchi sistemi che salvano solo le righe cambiate, Git scatta una **fotografia (snapshot)** dell'intero progetto ogni volta che fai un commit.

Risparmio di spazio

Se un file non è cambiato, Git crea un collegamento alla versione precedente per risparmiare spazio.

Integrità

Ogni stato è identificato da un hash univoco. Se un bit cambia, l'hash cambia (fondamentale per rilevare manomissioni).

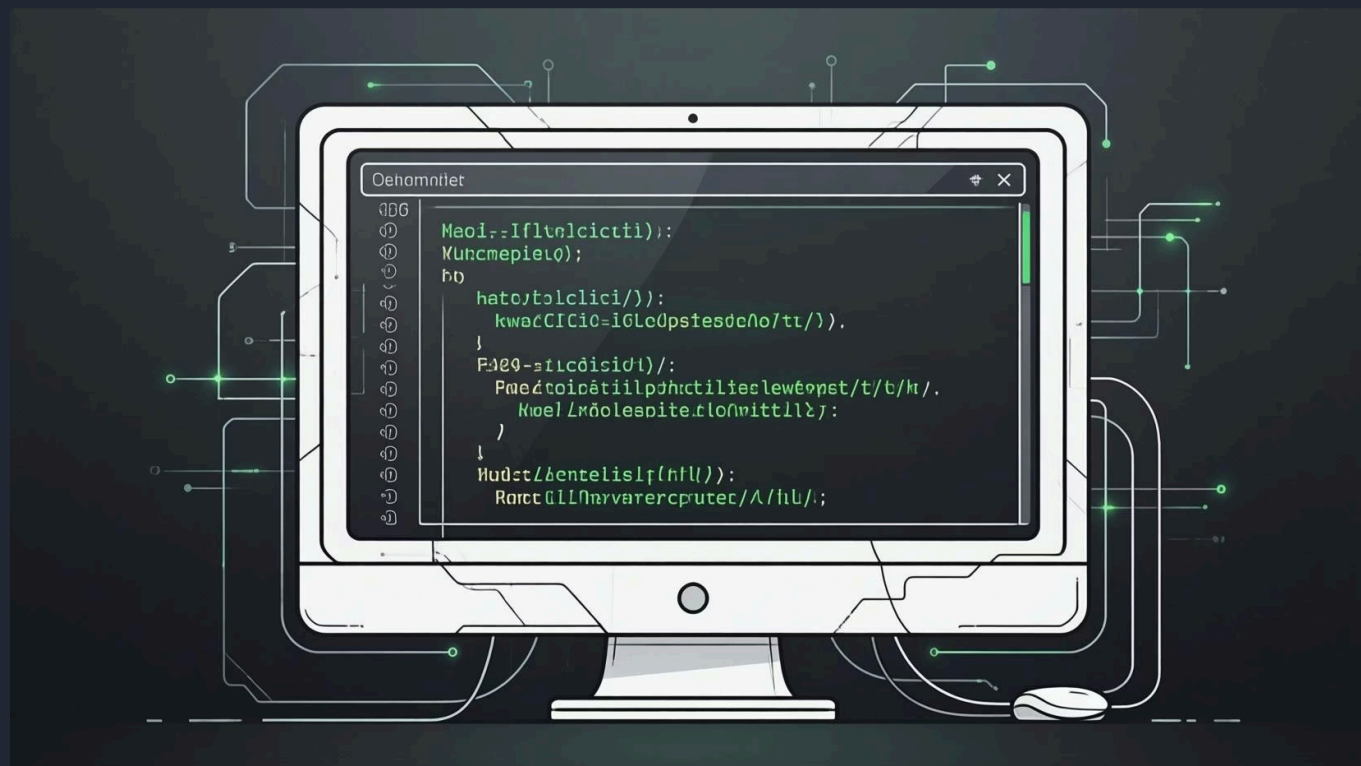


Configurazione e Inizializzazione

I primi passi per mettere in sicurezza l'identità dell'autore:

Comandi essenziali

- `git config --global user.name "Nome Cognome"`
- `git config --global user.email "email@esempio.com"`
- `git init` — Crea un nuovo repository locale (nasce la cartella nascosta `.git`).
- `git status` — Il comando più usato. Ti dice "dove sei" e cosa sta succedendo.



Il Ciclo di Vita del Codice

I comandi base per tracciare il lavoro:

01

```
git add <file>
```

Sposta i file dalla Working Directory allo Staging.

02

```
git commit -m "messaggio"
```

Crea lo snapshot definitivo.

Best Practice: Scrivi messaggi chiari, sono i tuoi log di controllo.

03

```
git log
```

Visualizza la cronologia degli snapshot (chi ha fatto cosa e quando).

Branching – Lavorare in parallelo

Cos'è un Branch?

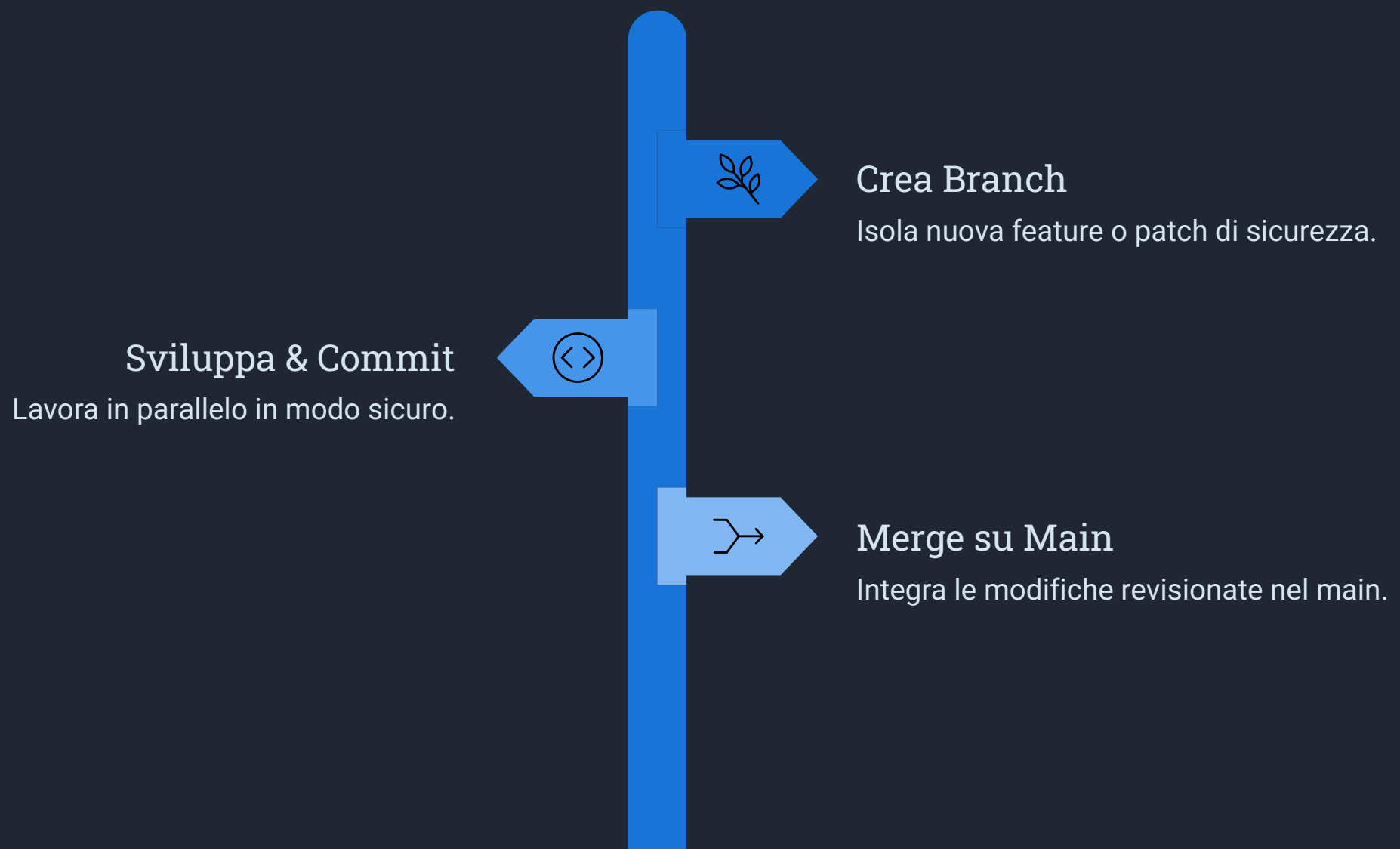
Una ramificazione del progetto (es. per testare una nuova patch di sicurezza senza rompere il codice principale).

Merge

Unire le modifiche del branch nel ramo principale (main).

Comandi chiave

- `git branch <nome>` – Crea un nuovo ramo.
- `git checkout <nome>` – Salta da un ramo all'altro.



Il branching è fondamentale per mantenere il codice principale stabile mentre si sperimentano nuove funzionalità o patch di sicurezza.

Collaborazione e Remote

Come interagire con server esterni (GitHub/GitLab):

```
git clone <url>
```

Copia un repository remoto sul tuo PC.

```
git remote add origin <url>
```

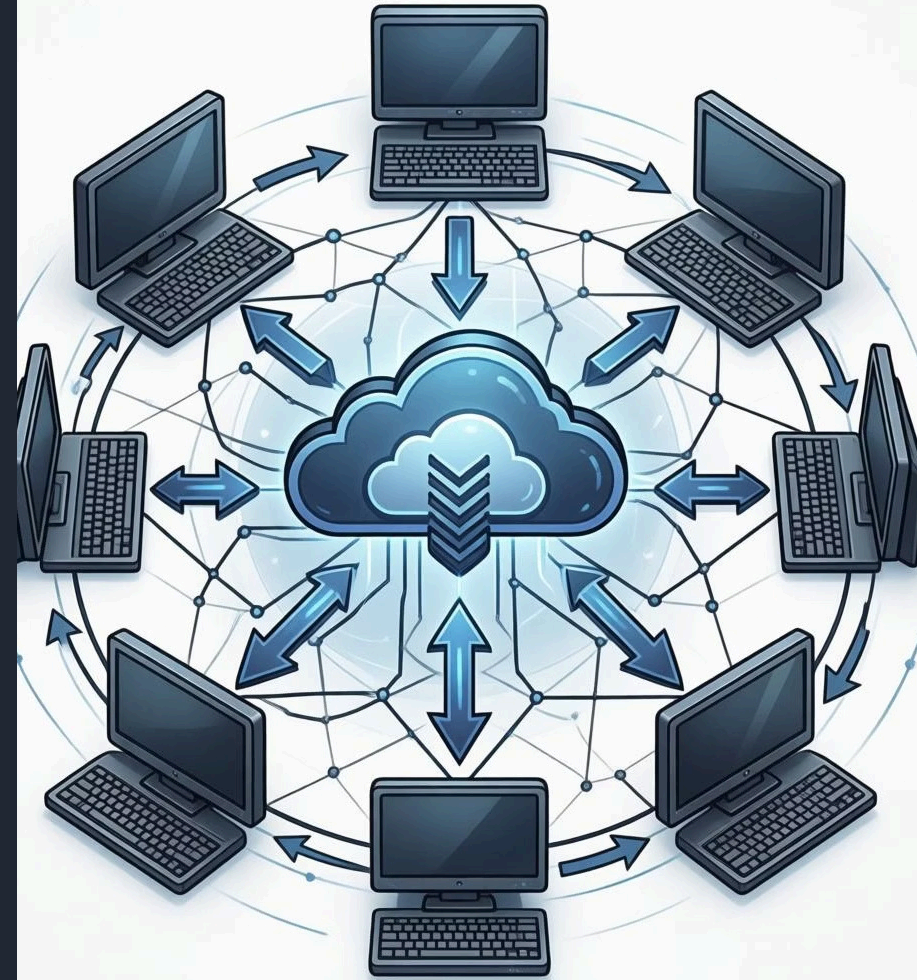
Collega il tuo repo locale a uno remoto.

```
git push
```

Invia i tuoi commit al server.

```
git pull
```

Scarica e unisce le modifiche dei colleghi sul tuo PC.



Security Pills per Principianti



Mai pushare segreti

Non includere chiavi API, password o certificati nei commit (usa `.gitignore`).



Il file `.gitignore`

Un file di testo dove elenchi cosa Git deve ignorare (es. cartelle `node_modules`, file `.env`, `log`).



SSH vs HTTPS

Preferisci le chiavi SSH per l'autenticazione remota per evitare di inserire password in chiaro.

- ❏ Certamente. Ecco altre due slide focalizzate su **GitHub** (e la collaborazione sicura), perfette per completare il set per gli studenti di L1.

GitHub – Oltre il semplice hosting



Social Coding

GitHub è la piattaforma cloud che ospita i repository Git e abilita la collaborazione globale.



Pull Requests (PR)

Il cuore della sicurezza collaborativa. Invece di scrivere direttamente sul codice principale, proponi una modifica che deve essere **revisionata e approvata** da altri (Code Review).



Issues

Un sistema di ticketing integrato per segnalare bug, vulnerabilità o nuove feature.



GitHub Actions

Automazione (CI/CD) per testare il codice e scansionare automaticamente eventuali falle di sicurezza a ogni invio.

Security & Collaboration Features

Per un futuro esperto di Cybersecurity, GitHub offre strumenti essenziali:



Dependabot

Scansiona le tue dipendenze (librerie esterne) e ti avvisa se sono vulnerabili, proponendo una patch automatica.



Secret Scanning

Ti blocca se provi a fare il *push* di una chiave privata o di una password (evitando leak disastrosi).



Forking Workflow

Copia un progetto altrui nel tuo account per sperimentare in isolamento senza sporcare il progetto originale.



Organizzazioni e Permessi

Gestione granulare di chi può leggere, scrivere o amministrare un repository (Principio del *Least Privilege*).